Tarea 2: Métricas sobre un procesador RISC-V

Accesos a memoria, saltos y operaciones entre registros en un pipeline de 5 etapas

Jorge Pablo Porras Alvarado
Josué Adrián Pérez Pérez
Marcelo Matey Rodríguez

13 de noviembre de 2026

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE 1107 — Fundamentos de Arquitectura de Computadores
Profesor: Luis Chavarría Zamora

1. Introducción

El desempeño de un procesador con pipeline depende del tipo de instrucciones que ejecuta y de cómo resuelve las dependencias entre ellas. Esta tarea mide, con el simulador Ripes [1], tres aspectos del funcionamiento de un procesador RISC-V: la cantidad de accesos a memoria, los saltos y las operaciones entre registros. Para cada aspecto se diseña un programa en ensamblador que realiza miles de operaciones de ese tipo, se cuantifican de forma analítica y se comprueban con la información de ejecución (*Execution info*) del simulador.

Cada programa se ejecuta en dos procesadores de 5 etapas de Ripes:

1. un pipeline **con** unidad de forwarding y detección de hazards, y
2. un pipeline **sin** ninguno de los dos, donde los riesgos se resuelven por software mediante transformación de código.

El documento presenta primero el marco teórico, luego el desarrollo de cada programa, después la discusión de los resultados y, finalmente, las conclusiones.

2. Marco teórico

Accesos a memoria. En un procesador con pipeline de cinco etapas (IF, ID, EX, MEM, WB), cada instrucción lee la memoria de instrucciones en la etapa IF, y solo las cargas y los almacenamientos acceden a la memoria de datos, en la etapa MEM [2, 3]. Por lo tanto, el número de accesos a la memoria de datos de un programa es igual al número de `lw`/`sw` ejecutados, y el de accesos a la memoria de instrucciones es al menos el número de instrucciones retiradas.

Rendimiento. El tiempo de ejecución de un programa se descompone como

$$T = N_{\text{instr}} \times \text{CPI} \times T_c, \tag{1}$$

donde N_{instr} es el número de instrucciones, CPI los ciclos por instrucción y T_c el periodo del reloj [2, 3]. En un pipeline ideal el CPI tiende a 1, y cada atasco o burbuja lo aumenta.

Hazards de datos. Una instrucción escribe su resultado en el banco de registros en la etapa WB (etapa 5), pero la instrucción siguiente lee sus operandos en ID (etapa 2). Si la que lee es una instrucción cercana a la que escribe (dependencia *read after write*, RAW), puede leer el valor viejo. Las dependencias WAR y WAW no causan problemas en este pipeline, porque las instrucciones avanzan en orden. El forwarding resuelve las RAW enviando el resultado directamente a la ALU sin esperar a WB. Si no existe ese hardware, hay que dejar una distancia suficiente entre las dos instrucciones, insertando `nop` (`addi zero, zero, 0`) o reordenando código. En el modelo de Ripes utilizado, esa distancia mínima resultó ser de 3 instrucciones, es decir, 2 `nop` entre dos instrucciones dependientes consecutivas (se verifica experimentalmente en la sección 3.1).

Hazards de control. Un salto tomado no se resuelve hasta la etapa EX, de modo que para entonces ya se buscaron instrucciones de más que deben descartarse. Eso introduce una penalización de ciclos por cada salto tomado, que el forwarding no elimina. En los resultados de la sección 3.1.3 se observa una penalización de 2 ciclos por salto tomado.

Saltos. RISC-V tiene dos tipos de instrucciones que cambian el flujo del programa: los saltos *condicionales* (`beq`, `bne`, `blt`, `bge`, etc.), que saltan solo si se cumple una comparación entre dos registros, y los saltos *incondicionales* (`jal`, `jalr`), que siempre saltan [2, 3]. Un salto condicional puede ser *tomado* o *no tomado*; uno incondicional siempre es tomado.

Mientras un salto se resuelve en EX, el procesador sigue buscando las instrucciones que vienen después en memoria, es decir, se comporta como si predijera que el salto *no* se toma. Si el salto no se toma, esas instrucciones son las correctas y no se pierde ningún ciclo; si se toma, hay que descartarlas. Por lo tanto, la penalización depende solo de los saltos tomados [3]. Las técnicas de predicción de saltos buscan reducir esa penalización adivinando el resultado antes de EX, a partir del sentido del salto o de su historial de ejecución [4].

Operaciones entre registros. RISC-V es una arquitectura de carga y almacenamiento (*load/store*): las operaciones aritméticas y lógicas solo operan sobre registros, y únicamente `lw` y `sw` acceden a la memoria de datos [2, 3, 5]. Las instrucciones de tipo R (`add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`, `slt`) leen dos registros fuente en ID, calculan el resultado en la ALU durante EX, no realizan ningún acceso en MEM y escriben el registro destino en WB. Las instrucciones con inmediato (`addi`, `andi`) combinan un registro con una constante, por lo que en esta tarea no se cuentan como operaciones entre registros.

Un programa formado solo por instrucciones de tipo R independientes entre sí es el mejor caso del pipeline: cada ciclo entra una instrucción y sale otra, y el CPI tiende a 1 [3]. El costo aparece cuando una instrucción usa el resultado de la anterior (dependencia RAW). Con forwarding, el resultado que la ALU calculó al final de EX se reenvía a la entrada de la ALU de la instrucción siguiente, desde los registros de segmentación EX/MEM y MEM/WB, de modo que una dependencia entre dos operaciones de ALU consecutivas no produce ningún atasco [2, 3]. Sin ese hardware, cada dependencia exige separar las instrucciones al menos 3 posiciones, es decir, 2 `nop` en el peor caso, como se determinó en la sección 3.1.

3. Desarrollo

3.1. Parte 1: Accesos a memoria

3.1.1. Programa

El programa recorre un arreglo de $N = 2000$ palabras y guarda en cada posición el valor de su propio índice. Para forzar tanto escrituras como lecturas, después de cada `sw` se hace un `lw` de la misma posición. El código final, con la cuantificación de accesos en los comentarios, es:

Listing 1: Programa de accesos a memoria (versión para el procesador con hazard unit).

```
    addi s2, zero, 0      # idx: i = 0
    addi t2, zero, 2000   # N = 2000
    lui  s1, 0x10000      # base = 0x10000000 (inicio del segmento de datos)
    lui  t0, 0x00000      # no se usa, slli lo sobrescribe (se deja tal como se corrio)

FOR:
    bge  s2, t2, done     # salta si i >= N: 2001 evaluaciones, solo la ultima salta
    slli t0, s2, 2        # t0 = 4*i (operacion entre registros)
    add  t0, t0, s1       # t0 = base + 4*i (operacion entre registros)
    sw   s2, 0(t0)        # ESCRITURA a memoria de datos: 1 por vuelta x 2000 = 2000
    lw   s3, 0(t0)        # LECTURA de memoria de datos: 1 por vuelta x 2000 = 2000
    addi s2, s2, 1        # i = i + 1
    jal  zero, FOR        # salto incondicional: 2000 veces

done:
    li   a0, 10           # syscall exit
    ecall                 # accesos totales a memoria de datos = 2000 + 2000 = 4000
```

Cuantificación. Cada vuelta ejecuta 7 instrucciones. Con las 4 de inicio y las 3 del final (el `bge` que sale del ciclo, `li` y `ecall`):

$$N_{\text{instr}} = 4 + 7 \times 2000 + 3 = 14007, \tag{2}$$

$$N_{\text{datos}} = \underbrace{2000}_{\texttt{sw}} + \underbrace{2000}_{\texttt{lw}} = 4000. \tag{3}$$

Además, cada instrucción retirada implica al menos una lectura de la memoria de instrucciones, así que hay al menos 14007 lecturas de instrucción contra 4000 accesos a datos.

Por qué funciona. Cada palabra ocupa 4 bytes, de modo que la dirección del elemento i es

$$\text{dir}(i) = \text{base} + 4i. \tag{4}$$

El desplazamiento a la izquierda $i \ll 2$ equivale a multiplicar por 4, que es lo que hace `slli t0, s2, 2`. La base `0x10000000` y los desplazamientos son múltiplos de 4, por lo que todos los accesos de palabra están alineados. La condición `bge s2, t2, done` se evalúa al inicio de cada vuelta, así que el cuerpo se ejecuta exactamente mientras $i < 2000$, es decir, 2000 veces, y la condición se evalúa 2001 veces.

Verificación. Al terminar, la memoria debe contener $0, 1, \ldots, 1999$ en palabras consecutivas a partir de `0x10000000`. El último elemento está en

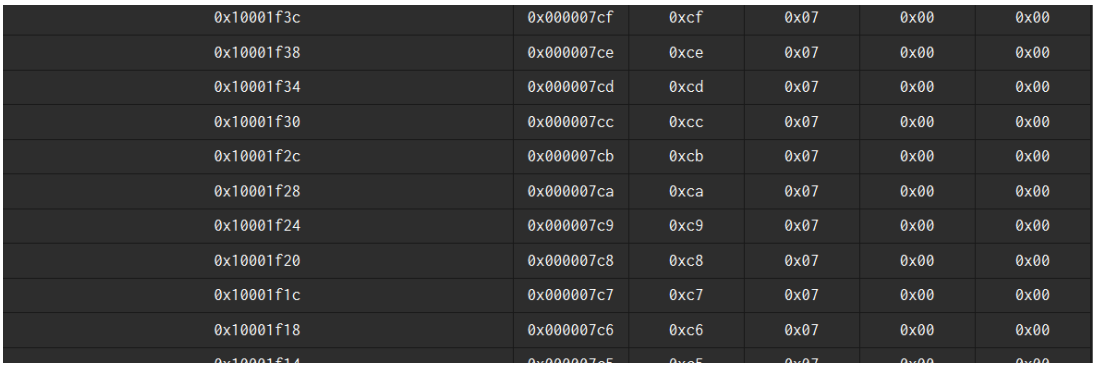
$$\texttt{0x10000000} + 4 \times 1999 = \texttt{0x10001F3C}, \tag{5}$$

y la dirección `0x10001F40`, que sería el elemento 2000, debe seguir en cero porque el ciclo se detuvo antes de escribirla.

3.1.2. Versión para el procesador sin hardware de riesgos

En el modelo sin forwarding ni detección de hazards, las dependencias RAW entre instrucciones cercanas producirían resultados incorrectos. La solución es la transformación de código: insertar `nop` hasta que la instrucción que lee quede a una distancia de al menos 3 instrucciones de la que escribe. La Tabla 1 resume los pares RAW del programa.

Algunas observaciones sobre la tabla:



0x10001f3c	0x000007cf	0xcf	0x07	0x00	0x00
0x10001f38	0x000007ce	0xce	0x07	0x00	0x00
0x10001f34	0x000007cd	0xcd	0x07	0x00	0x00
0x10001f30	0x000007cc	0xcc	0x07	0x00	0x00
0x10001f2c	0x000007cb	0xcb	0x07	0x00	0x00
0x10001f28	0x000007ca	0xca	0x07	0x00	0x00
0x10001f24	0x000007c9	0xc9	0x07	0x00	0x00
0x10001f20	0x000007c8	0xc8	0x07	0x00	0x00
0x10001f1c	0x000007c7	0xc7	0x07	0x00	0x00
0x10001f18	0x000007c6	0xc6	0x07	0x00	0x00

Figura 1: Pestaña Memory de Ripes tras la ejecución: el último elemento, `0x000007CF` = 1999, está en `0x10001F3C`, y las palabras anteriores decrecen de uno en uno.

Escribe	Lee	Reg.	Dist. original	`nop`	Dist. final
`slli t0`	`add t0`	`t0`	1	2	3
`add t0`	`sw 0(t0)`	`t0`	1	2	3
`add t0`	`lw 0(t0)`	`t0`	2	(los mismos 2)	4
`addi s2`	`bge` (sig. vuelta)	`s2`	2	2	4
`addi s2`	`slli` (sig. vuelta)	`s2`	3	0	5
`addi t2` (inicio)	primer `bge`	`t2`	3	0	3
`addi s2` (inicio)	primer `bge`	`s2`	4	0	4

Tabla 1: Dependencias RAW y `nop` colocados. La distancia es el número de posiciones entre las dos instrucciones en el orden del programa.

- El `sw` y el `lw` de la misma dirección no generan un hazard de registros, porque las dos instrucciones acceden a memoria en orden.
- Los dos `nop` antes del `jal` se colocaron para garantizar la distancia mínima entre `addi s2` y el `bge` de la vuelta siguiente en el orden del programa, sin depender de los ciclos de burbuja que introduce el salto.
- Los pares de inicio con distancia 3 y 4 funcionaron sin `nop`: el primer `bge` lee `t2` a distancia 3 y el programa llenó la memoria correctamente. Si hubiera leído el valor anterior (cero), habría comparado $0 \geq 0$ y habría salido sin escribir nada. Esto confirma que la distancia 3 es suficiente en este modelo.
- Tras el `jal` no se colocaron `nop`. El comportamiento observado es consistente con que el procesador descarta las instrucciones buscadas después de un salto tomado (sección 3.1.3).
- El registro `s3` cargado por `lw` no se utiliza después, por lo que este programa no presenta el hazard de carga seguida de uso.

El código transformado es:

Listing 2: Versión con `nop` para el procesador sin forwarding ni detección de hazards.

```
    addi s2, zero, 0      # idx: i = 0
    addi t2, zero, 2000   # N = 2000
    lui  s1, 0x10000      # base = 0x10000000
    lui  t0, 0x00000      # no se usa

FOR:
    bge  s2, t2, done     # salta si i >= N
    slli t0, s2, 2        # t0 = 4*i
    nop                   # RAW slli -> add: se necesitan 2 nop
```

```
    nop
    add  t0, t0, s1       # t0 = base + 4*i
    nop                   # RAW add -> sw y add -> lw: se necesitan 2 nop
    nop
    sw   s2, 0(t0)        # ESCRITURA: 2000 en total
    lw   s3, 0(t0)        # LECTURA: 2000 en total
    addi s2, s2, 1        # i = i + 1
    nop                   # distancia minima hasta el bge de la vuelta siguiente
    nop
    jal  zero, FOR        # salto incondicional

done:
    li   a0, 10
    ecall
```

Esta versión ejecuta 13 instrucciones por vuelta (7 útiles y 6 `nop`), por lo que

$$N_{\text{instr}} = 4 + 13 \times 2000 + 3 = 26007. \tag{6}$$

Los `nop` no tocan la memoria de datos, así que los accesos a datos siguen siendo 4000.

3.1.3. Resultados

La Tabla 2 reporta la información de ejecución (Execution info) de Ripes para cada procesador.

Métrica	Sin hazard (con `nop`)	Con hazard (original)
Ciclos	30013	18013
Instrucciones retiradas	26007	14007
CPI	1.15	1.29
IPC	0.867	0.778
`nop` ejecutados	12000	0
Accesos a memoria de datos	4000	4000
Saltos tomados	2001	2001

Tabla 2: Métricas obtenidas en Ripes para el programa de accesos a memoria en ambos procesadores.

Modelo de ciclos. En ambos procesadores los ciclos se ajustan exactamente a

$$C = N_{\text{instr}} + 4 + 2\,N_{\text{saltos}}, \tag{7}$$

donde 4 ciclos son el llenado del pipeline y N_{saltos} cuenta los saltos tomados. Con 2001 saltos tomados (los 2000 `jal` más el `bge` final):

$$\text{con hazard:} \quad 14007 + 4 + 2 \times 2001 = 18013, \tag{8}$$

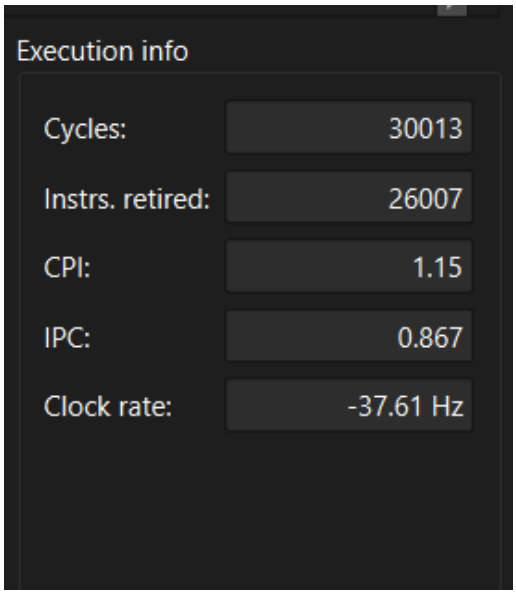
$$\text{sin hazard:} \quad 26007 + 4 + 2 \times 2001 = 30013. \tag{9}$$

Esto es consistente con una penalización de 2 ciclos por salto tomado, la que corresponde a descartar las 2 instrucciones buscadas antes de que el salto se resuelva en EX.

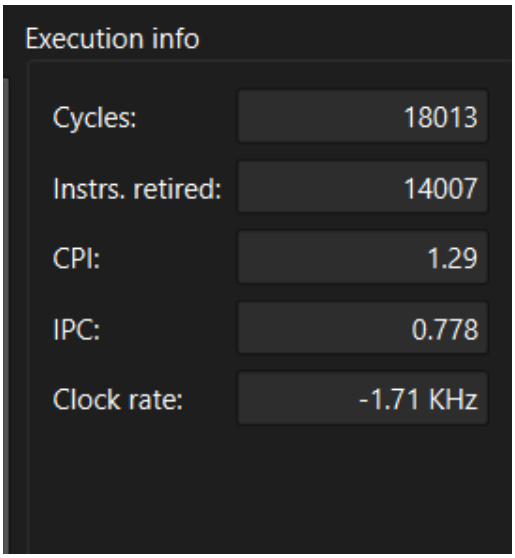
3.2. Parte 2: Saltos

3.2.1. Programa

Para comprobar que solo los saltos tomados tienen costo, el programa debe ejecutar saltos tomados y no tomados en cantidades conocidas. Se recorre un ciclo de $N = 2000$ vueltas y, en cada



(a) Sin hazard (con `nop`)



(b) Con hazard (original)

Figura 2: Execution info de Ripes en cada procesador. El campo Clock rate no se utiliza en el análisis.

una, un salto condicional decide si el índice i es par o impar: si es par, salta a sumar un contador de pares; si es impar, no salta, suma un contador de impares y luego salta incondicionalmente al final del cuerpo. La condición del ciclo se evalúa al final de cada vuelta. El código final es:

Listing 3: Programa de saltos (versión para el procesador con hazard unit).

```
    addi s2, zero, 0 #idx: i = 0
    addi t2, zero, 2000 #N = 2000
    addi s1, zero, 0 # contador impares
    addi s0, zero, 0 # contador pares
LOOP:
    andi t0, s2, 1 # se ejecuta 2000 veces
    beq t0, zero, PAR # se ejecuta 2000 veces, se toma el salto 1000 veces y no se toma 1000
     veces
    addi s1, s1, 1 # suma impares, se ejecuta 1000 veces
    jal zero, SIGUE # se toma este salto 1000 veces
PAR:
    addi s0, s0, 1 # suma pares, se ejecuta 1000 veces
SIGUE:
    addi s2, s2, 1 # se ejecuta 2000 veces
    blt s2, t2, LOOP # se toma el salto 1999 veces y no se toma 1 vez
done:
    li a7, 10 # syscall exit
    nop
    nop
    nop
    ecall
# se ejecutan 5000 saltos, de los cuales solo se toman 3999 y no se toman 1001
```

Los tres `nop` antes del `ecall` no forman parte de la medición de saltos; su justificación se da en la sección 3.2.3.

Cuantificación. La Tabla 3 resume cuántas veces se ejecuta cada salto y cuántas veces se toma.

Las vueltas pares ejecutan 5 instrucciones (`andi`, `beq`, `addi s0`, `addi s2`, `blt`) y las impares 6, porque se agregan `addi s1` y `jal`. Con las 4 instrucciones de inicio y las 5 del final (`li`, tres

Instrucción	Ejecuciones	Tomados	No tomados
`beq t0, zero, PAR`	2000	1000	1000
`jal zero, SIGUE`	1000	1000	0
`blt s2, t2, LOOP`	2000	1999	1
Total	5000	3999	1001

Tabla 3: Cuantificación de los saltos del programa.

`nop` y `ecall`):

$$N_{\text{instr}} = 4 + 1000 \times 5 + 1000 \times 6 + 5 = 11009, \tag{10}$$

$$N_{\text{tomados}} = 1000 + 1000 + 1999 = 3999. \tag{11}$$

Por qué funciona. La instrucción `andi t0, s2, 1` conserva solo el bit menos significativo de i, que vale 0 si i es par y 1 si es impar. Como i recorre los valores $0, 1, \ldots, 1999$, hay exactamente 1000 pares y 1000 impares, de modo que el `beq` se toma 1000 veces y no se toma otras 1000. El `jal` solo se ejecuta en el camino impar, así que se ejecuta 1000 veces y siempre se toma. Como la condición del ciclo está al final, el cuerpo se ejecuta para $i = 0$ antes de evaluar `blt`; después de incrementar, `blt` compara $i + 1 < 2000$, que se cumple 1999 veces y falla una sola vez, cuando $i + 1 = 2000$.

Verificación. Al terminar, los contadores deben valer `s0` = `s1` = 1000 = `0x3E8`, lo que demuestra que cada camino del `beq` se ejecutó las veces calculadas.

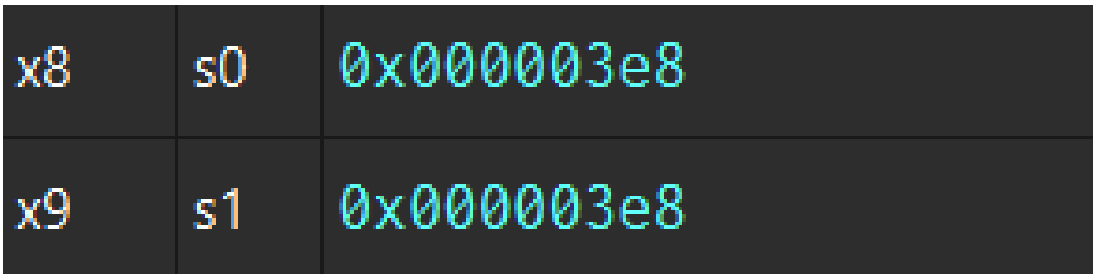


Figura 3: Estado de los registros `s0` y `s1` al concluir el programa.

3.2.2. Versión para el procesador sin hardware de riesgos

Los saltos también leen registros, por lo que participan en dependencias RAW. La Tabla 4 resume los pares del programa y los `nop` insertados con la misma regla de la Parte 1: una distancia mínima de 3 instrucciones entre la que escribe y la que lee.

Los contadores `s0` y `s1` solo se vuelven a leer en vueltas posteriores, muy lejos de su escritura. Después de los saltos tomados no se colocaron `nop`, porque el procesador descarta las instrucciones buscadas en el camino equivocado (sección 3.1.3). El código transformado es:

Listing 4: Versión con `nop` para el procesador sin forwarding ni detección de hazards.

```
    addi s2, zero, 0 #idx: i = 0
    addi t2, zero, 2000 #N = 2000
    addi s1, zero, 0 # contador impares
    addi s0, zero, 0 # contador pares
LOOP:
    andi t0, s2, 1 # se ejecuta 2000 veces
    nop # RAW andi -> beq (t0): 2 nop
```

Escribe	Lee	Reg.	Dist. original	`nop`	Dist. final
`andi t0`	`beq t0`	`t0`	1	2	3
`addi s2` (SIGUE)	`blt s2`	`s2`	1	2	3
`addi s2` (SIGUE)	`andi` (sig. vuelta)	`s2`	2	(los mismos 2)	4
`addi s2` (inicio)	primer `andi`	`s2`	4	0	4
`addi s0` (inicio)	primer `addi s0`	`s0`	5	0	5
`li a7`	`ecall`	`a7`	1	3	4

Tabla 4: Dependencias RAW del programa de saltos y `nop` colocados. La distancia se mide en el orden de ejecución.

```
    nop
    beq t0, zero, PAR # se ejecuta 2000 veces, se toma el salto 1000 veces y no se toma 1000
    veces
    addi s1, s1, 1 # suma impares, se ejecuta 1000 veces
    jal zero, SIGUE # se toma este salto 1000 veces
PAR:
    addi s0, s0, 1 # suma pares, se ejecuta 1000 veces
SIGUE:
    addi s2, s2, 1 # se ejecuta 2000 veces
    nop # RAW addi -> blt (s2): 2 nop
    nop
    blt s2, t2, LOOP # se toma el salto 1999 veces y no se toma 1 vez, se ejecuta 2000 veces
done:
    li a7, 10 # syscall exit
    nop
    nop
    nop
    ecall
# se ejecutan 5000 saltos, de los cuales solo se toman 3999 y no se toman 1001
```

Se agregan 4 `nop` por vuelta, así que

$$N_{\text{instr}} = 11009 + 4 \times 2000 = 19009. \tag{12}$$

Los `nop` no son saltos, por lo que la cuantificación de la Tabla 3 no cambia.

3.2.3. Terminación del programa: el `ecall`

En Ripes, el número de llamada al sistema se pasa en `a7`; con `a7` $= 10$, `ecall` termina el programa. Al probar distintas formas de terminar se encontró que el `ecall` tiene un costo y un comportamiento propios, resumidos en la Tabla 5.

Procesador	Antes del `ecall`	Resultado	Ciclos extra
Con hazard unit	`li a7` a distancia 1	termina bien	2
Con hazard unit	`li a7` + 3 `nop`	termina bien	2
Con hazard unit	`blt` (`li a7` al inicio)	termina bien	1
Sin hazard unit	`li a7` + 3 `nop`	termina bien	0
Sin hazard unit	`blt` (`li a7` al inicio)	termina tras 1 vuelta	—

Tabla 5: Comportamiento del `ecall` de salida según el procesador y lo que lo precede. Los ciclos extra son los que exceden el modelo de la ecuación (13).

El código fuente de Ripes explica estos resultados [6]:

- **Con hazard unit**, el `ecall` se atiende en la etapa EX. Como depende implícitamente de todos los registros, la unidad de hazards lo detiene mientras alguna instrucción en MEM o

WB tenga una escritura pendiente al banco de registros. Si las dos instrucciones anteriores escriben un registro, el atasco es de 2 ciclos; si solo la segunda anterior escribe (el `blt` no escribe), es de 1 ciclo. Los `nop` (`addi zero, zero, 0`) cuentan como escrituras, por lo que no evitan el atasco.

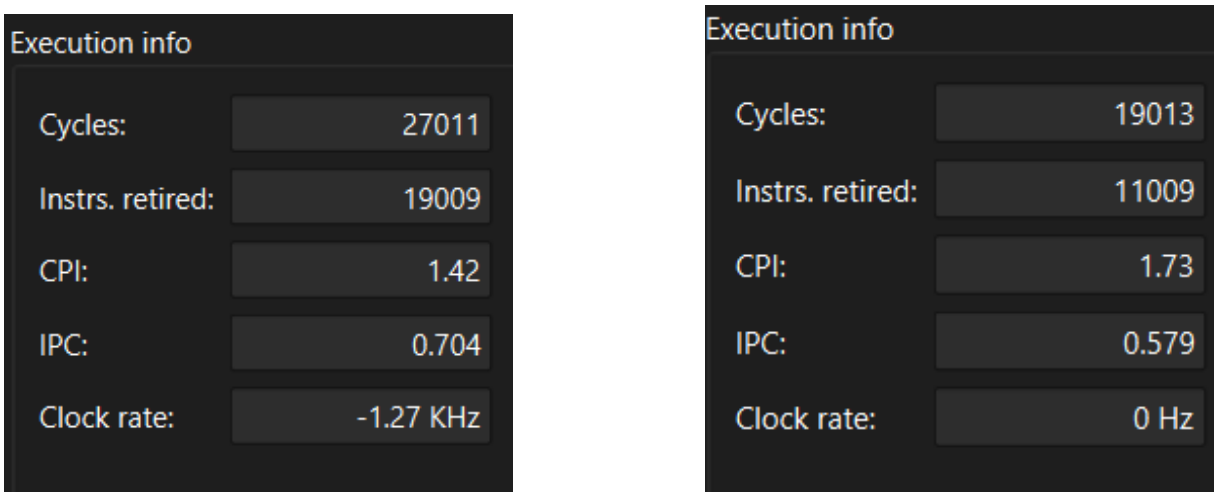
- **Sin hazard unit**, el `ecall` se atiende en la etapa ID, sin ningún atasco. Por eso necesita que `li a7` esté a distancia 4 (tres `nop`). Además, si el `ecall` está justo después del `blt`, llega a ID en el mismo ciclo en que el `blt` se resuelve en EX y se ejecuta antes de ser descartado. Con `a7` ya cargado desde el inicio, el programa terminó al final de la primera vuelta, con `s0` $= 1$ y `s1` $= 0$.

El último caso es un hazard de control sobre una instrucción con efectos fuera del pipeline: una instrucción buscada en el camino equivocado de un salto no debería tener efectos hasta que el salto se confirme. Por estas razones, ambas versiones finales terminan con `li a7, 10` seguido de tres `nop` antes del `ecall`.

3.2.4. Resultados

Métrica	Sin hazard (con `nop`)	Con hazard (original)
Ciclos	27011	19013
Instrucciones retiradas	19009	11009
CPI	1.42	1.73
IPC	0.704	0.579
`nop` ejecutados	8003	3
Saltos ejecutados	5000	5000
Saltos tomados / no tomados	3999 / 1001	3999 / 1001

Tabla 6: Métricas obtenidas en Ripes para el programa de saltos en ambos procesadores.



(a) Sin hazard (con `nop`)

(b) Con hazard (original)

Figura 4: Execution info de Ripes para el programa de saltos. El campo Clock rate no se utiliza en el análisis.

Modelo de ciclos. Al modelo de la Parte 1 se le agrega el atasco del `ecall`, S_{ecall}:

$$C = N_{\text{instr}} + 4 + 2\,N_{\text{tomados}} + S_{\text{ecall}}, \tag{13}$$

con $S_{\text{ecall}} = 2$ en el procesador con hazard unit y 0 en el otro (sección 3.2.3). Con $N_{\text{tomados}} = 3999$:

$$\text{con hazard:} \quad 11009 + 4 + 2 \times 3999 + 2 = 19013, \tag{14}$$

$$\text{sin hazard:} \quad 19009 + 4 + 2 \times 3999 + 0 = 27011, \tag{15}$$

que coinciden exactamente con lo medido. Ninguno de los 1001 saltos no tomados aparece en el modelo.

3.3. Parte 3: Operaciones entre registros

3.3.1. Programa

El programa ejecuta un ciclo de $N = 2000$ vueltas. En cada vuelta se realizan 8 operaciones de tipo R, y cada una usa el resultado de la anterior, de modo que forman una cadena de dependencias RAW. Los operandos `t0` y `t1` se incrementan al final de cada vuelta para que los resultados cambien. Los incrementos y el contador del ciclo son instrucciones `addi` (tipo I), por lo que no se cuentan como operaciones entre registros. El código final se muestra en el Listado 5:

Listing 5: Programa de operaciones entre registros (versión para el procesador con hazard unit).

```
    addi s2, zero, 0      # idx: i = 0
    addi t0, zero, 3      # operando A = 3
    addi t1, zero, 5      # operando B = 5
    addi t2, zero, 2000   # N = 2000

LOOP:
    addi s2, s2, 1        # i = i + 1, se ejecuta 2000 veces (tipo I, no cuenta)
    add  s3, t0, t1       # operacion entre registros #1, se ejecuta 2000 veces
    sub  s4, s3, t0       # operacion entre registros #2, se ejecuta 2000 veces (RAW con s3)
    and  s5, s4, t1       # operacion entre registros #3, se ejecuta 2000 veces (RAW con s4)
    or   s6, s5, t0       # operacion entre registros #4, se ejecuta 2000 veces (RAW con s5)
    xor  s7, s6, t1       # operacion entre registros #5, se ejecuta 2000 veces (RAW con s6)
    sll  s8, s7, t0       # operacion entre registros #6, se ejecuta 2000 veces (RAW con s7)
    srl  s9, s8, t1       # operacion entre registros #7, se ejecuta 2000 veces (RAW con s8)
    slt  s10, s9, t0      # operacion entre registros #8, se ejecuta 2000 veces (RAW con s9)
    addi t0, t0, 1        # A = A + 1, se ejecuta 2000 veces (tipo I, no cuenta)
    addi t1, t1, 2        # B = B + 2, se ejecuta 2000 veces (tipo I, no cuenta)
    blt  s2, t2, LOOP     # se ejecuta 2000 veces, se toma el salto 1999 veces y no se toma 1
    vez

done:
    li   a7, 10           # syscall exit
    ecall
# se ejecutan 8 operaciones entre registros por vuelta x 2000 vueltas = 16000 operaciones
# instrucciones totales: 4 (inicio) + 2000 x 12 (lazo) + 2 (salida) = 24006
# las 7 dependencias RAW por vuelta se resuelven con el forwarding de la hazard unit, sin nop
```

Cuantificación. Cada vuelta ejecuta 12 instrucciones: 8 de tipo R, 3 `addi` y el `blt`. Con las 4 instrucciones de inicio y las 2 del final (`li` y `ecall`):

$$N_{\text{instr}} = 4 + 12 \times 2000 + 2 = 24006, \tag{16}$$

$$N_{\text{R}} = 8 \times 2000 = 16000, \tag{17}$$

$$N_{\text{tomados}} = 1999. \tag{18}$$

Por qué funciona. Las instrucciones de tipo R no acceden a la memoria de datos ni cambian el flujo del programa, así que cada una se retira exactamente una vez por vuelta y su número de ejecuciones se puede contar directamente: 8 por vuelta, 16000 en total, que es un valor verificable contra las instrucciones retiradas de Ripes. Al encadenar las operaciones, cada una consume el

resultado de la anterior, lo que fuerza 7 dependencias RAW por vuelta (14000 en total) y permite observar el costo de resolverlas con y sin hardware. Los incrementos de `t0` y `t1` hacen que los resultados cambien en cada vuelta y que el procesador no repita siempre el mismo cálculo. La condición del ciclo está al final: después de incrementar, `blt` compara $i+1 < 2000$, que se cumple 1999 veces y falla una sola vez.

Verificación. En la última vuelta `t0` = 2002 y `t1` = 4003, y al terminar los registros deben valer lo indicado en la Tabla 7. La Figura 5 muestra los registros de Ripes al concluir la ejecución, y los resultados de las 8 operaciones coinciden en ambos procesadores.

Registro	Operación	Decimal	Hexadecimal
`s3`	`add`	6005	`0x1775`
`s4`	`sub`	4003	`0xFA3`
`s5`	`and`	4003	`0xFA3`
`s6`	`or`	4083	`0xFF3`
`s7`	`xor`	80	`0x50`
`s8`	`sll`	20971520	`0x1400000`
`s9`	`srl`	2621440	`0x280000`
`s10`	`slt`	0	`0x0`

Tabla 7: Valores finales esperados de los registros de la Parte 3.

3.3.2. Versión para el procesador sin hardware de riesgos

Con la misma regla de las partes anteriores (distancia mínima de 3 instrucciones entre la que escribe y la que lee), la Tabla 8 resume los pares RAW del programa.

Escribe	Lee	Reg.	Dist. original	`nop`	Dist. final
cada operación R (7 pares)	la siguiente de la cadena	`s3`–`s9`	1	2	3
`addi t1`	`add s3` (sig. vuelta)	`t1`	3	0	3
`addi t0`	`add s3` (sig. vuelta)	`t0`	4	0	4
`addi s2`	`blt`	`s2`	11	0	11
`addi t1` (inicio)	primer `add s3`	`t1`	3	0	3
`li a7`	`ecall`	`a7`	1	3	4

Tabla 8: Dependencias RAW del programa de operaciones entre registros y `nop` colocados.

Los incrementos de `t0` y `t1` y el contador `s2` se colocaron en posiciones que dejan distancia suficiente sin `nop`: los `addi` de `t0` y `t1` van al final del cuerpo y el `blt` y el `addi s2` de la vuelta siguiente hacen de relleno. En cambio, la cadena de operaciones no se puede separar reordenando, porque cada instrucción depende de la anterior, y requiere 2 `nop` por cada uno de sus 7 pares, es decir, 14 `nop` por vuelta. El código transformado se muestra en el Listado 6:

Listing 6: Versión con `nop` para el procesador sin forwarding ni detección de hazards.

```
    addi s2, zero, 0      # idx: i = 0
    addi t0, zero, 3      # operando A = 3
    addi t1, zero, 5      # operando B = 5
    addi t2, zero, 2000   # N = 2000

LOOP:
    addi s2, s2, 1        # i = i + 1, se ejecuta 2000 veces (tipo I, no cuenta)
    add  s3, t0, t1       # operacion entre registros #1, se ejecuta 2000 veces
    nop                   # RAW s3 -> sub: 2 nop
    nop
    sub  s4, s3, t0       # operacion entre registros #2, se ejecuta 2000 veces
```

```
    nop                 # RAW s4 -> and: 2 nop
    nop
    and  s5, s4, t1     # operacion entre registros #3, se ejecuta 2000 veces
    nop                 # RAW s5 -> or: 2 nop
    nop
    or   s6, s5, t0     # operacion entre registros #4, se ejecuta 2000 veces
    nop                 # RAW s6 -> xor: 2 nop
    nop
    xor  s7, s6, t1     # operacion entre registros #5, se ejecuta 2000 veces
    nop                 # RAW s7 -> sll: 2 nop
    nop
    sll  s8, s7, t0     # operacion entre registros #6, se ejecuta 2000 veces
    nop                 # RAW s8 -> srl: 2 nop
    nop
    srl  s9, s8, t1     # operacion entre registros #7, se ejecuta 2000 veces
    nop                 # RAW s9 -> slt: 2 nop
    nop
    slt  s10, s9, t0    # operacion entre registros #8, se ejecuta 2000 veces
    addi t0, t0, 1      # A = A + 1, se ejecuta 2000 veces (tipo I, no cuenta)
    addi t1, t1, 2      # B = B + 2, se ejecuta 2000 veces (tipo I, no cuenta)
    blt  s2, t2, LOOP   # s2 escrito lejos: sin RAW, se toma 1999 veces y no se toma 1 vez

done:
    li   a7, 10         # syscall exit
    nop                 # RAW a7 -> ecall: 3 nop
    nop
    nop
    ecall
# se ejecutan 8 operaciones entre registros por vuelta x 2000 vueltas = 16000 operaciones
# instrucciones totales: 4 (inicio) + 2000 x 26 (lazo: 12 + 14 nop) + 2 (salida) + 3 nop = 52009
# los 14 nop por vuelta (28000 en total) son el costo de resolver los 7 RAW por software
```

Esta versión ejecuta 26 instrucciones por vuelta (12 útiles y 14 `nop`), por lo que

$$N_{\text{instr}} = 4 + 26 \times 2000 + 5 = 52009. \tag{19}$$

Los `nop` no son operaciones entre registros de la cadena, así que $N_{\text{R}} = 16000$ en ambos procesadores.

3.3.3. Resultados

La Tabla 9 reporta la información de ejecución (Execution info) de Ripes para cada procesador, y la Figura 6 muestra las capturas correspondientes.

Métrica	Sin hazard (con `nop`)	Con hazard (original)
Ciclos	56011	28010
Instrucciones retiradas	52009	24006
CPI	1.08	1.17
IPC	0.929	0.857
`nop` ejecutados	28003	0
Operaciones entre registros	16000	16000
Saltos tomados	1999	1999

Tabla 9: Métricas obtenidas en Ripes para el programa de operaciones entre registros en ambos procesadores.

Modelo de ciclos. Los ciclos se ajustan al modelo de la Parte 2,

$$C = N_{\text{instr}} + 4 + 2\,N_{\text{tomados}} + S_{\text{ecall}}, \tag{20}$$

con $S_{\text{ecall}} = 2$ en el procesador con hazard unit (el `li a7` queda a distancia 1 del `ecall`) y 0 en el otro, y $N_{\text{tomados}} = 1999$:

$$\text{con hazard:} \quad 24006 + 4 + 2 \times 1999 + 2 = 28010, \tag{21}$$

$$\text{sin hazard:} \quad 52009 + 4 + 2 \times 1999 + 0 = 56011, \tag{22}$$

que coinciden exactamente con lo medido.

4. Discusión

4.1. Parte 1: Accesos a memoria

Los accesos a memoria no cambian. Los accesos a la memoria de datos son 4000 en ambos casos: la transformación por software agrega instrucciones, pero ninguna toca la memoria de datos. Lo que sí cambia es el tráfico de la memoria de instrucciones, que pasa de 14007 a 26007 lecturas como mínimo.

El CPI es engañoso. La versión con `nop` tiene mejor CPI (1.15 contra 1.29), pero tarda más. Los `nop` cuentan como instrucciones retiradas y reducen el promedio sin hacer trabajo útil. Contando solo las 14007 instrucciones útiles, su CPI efectivo es

$$\frac{30013}{14007} \approx 2{,}14. \tag{23}$$

Por eso la comparación correcta es en ciclos totales:

$$\frac{30013}{18013} \approx 1{,}67, \tag{24}$$

es decir, el procesador con hazard unit es 1.67 veces más rápido con el mismo programa, y la diferencia es $30013 - 18013 = 12000 = 6 \times 2000$ ciclos, exactamente un ciclo por cada `nop` ejecutado.

Hazards de datos contra hazards de control. Con forwarding, los hazards de datos de este programa se resuelven sin atascos: el único sobrecosto que queda son los $2 \times 2001 = 4002$ ciclos de los saltos, un 22 % del total en el procesador con hazard unit y un 13 % en el otro. Resolver los hazards de datos por software cuesta 12000 ciclos, un 40 % del tiempo de la versión con `nop`. Los hazards de control, en cambio, no desaparecen con ninguna de las dos soluciones, y requerirían técnicas como la predicción de saltos para reducirse [2, 3].

Limitación. Como el valor cargado por `lw` no se usa en la instrucción siguiente, este programa no exhibe el hazard de carga seguida de uso, que ni siquiera el forwarding resuelve sin un atasco de un ciclo. Un programa que consumiera de inmediato ese valor mostraría un costo adicional en el procesador con hazard unit y más `nop` en el otro.

4.2. Parte 2: Saltos

Solo los saltos tomados cuestan. Los ciclos que exceden a las instrucciones son

$$C - N_{\text{instr}} = 4 + 2 \times 3999 + S_{\text{ecall}}, \tag{25}$$

sin ningún término para los 1001 saltos no tomados. Esto confirma que el pipeline de Ripes continúa buscando instrucciones de forma secuencial mientras resuelve el salto, lo que equivale a predecir siempre "no tomado" [3]: cuando acierta, el salto no tiene costo; cuando falla, se pagan 2 ciclos.

Peso de los saltos en el tiempo de ejecución. Los saltos tomados imponen $2 \times 3999 = 7998$ ciclos en ambos procesadores, un 42 % de los 19013 ciclos con hazard unit y un 30 % de los 27011 sin ella. Ni el forwarding ni los `nop` reducen este costo, porque es un hazard de control y no de datos.

El CPI vuelve a ser engañoso. Igual que en la Parte 1, la versión con `nop` tiene mejor CPI (1.42 contra 1.73), pero tarda $27011/19013 \approx 1{,}42$ veces más. La diferencia de ciclos, $27011 - 19013 = 7998$, corresponde a los 8000 `nop` agregados en el ciclo menos los 2 ciclos del atasco del `ecall`, que solo ocurre con hazard unit.

Cómo se podría reducir el costo.

- **Estructura del ciclo.** Con la condición al final, el ciclo necesita una sola instrucción de control por vuelta (`blt`). El programa de la Parte 1, con la condición al inicio, necesita dos (`bge` y `jal`). Ambas formas pagan un salto tomado por vuelta, pero la de la Parte 1 ejecuta una instrucción más por vuelta, es decir, 2000 ciclos más en total.
- **Predicción de saltos.** El `blt` se toma en 1999 de 2000 ejecuciones, así que un predictor que suponga "tomado" para los saltos hacia atrás acertaría casi siempre. El `beq`, en cambio, alterna entre tomado y no tomado, y un predictor que repite el último resultado de cada salto fallaría en todas sus ejecuciones después de la primera [4].
- **Transformación de código.** Como la paridad de i alterna, desenrollar el ciclo dos veces (procesar un índice par y uno impar por vuelta) eliminaría el `beq` y el `jal`, es decir, al menos 2000 saltos tomados y 4000 ciclos de penalización, además de reducir a la mitad las ejecuciones del `blt` [3].

4.3. Parte 3: Operaciones entre registros

El forwarding resuelve las dependencias de ALU sin costo. El procesador con hazard unit retira 24006 instrucciones en 28010 ciclos. Los $28010 - 24006 = 4004$ ciclos de más se explican por completo con el llenado del pipeline (4), los saltos tomados ($2 \times 1999 = 3998$) y el atasco del `ecall` (2). Las 14000 dependencias RAW de la cadena no agregan ningún ciclo, porque el forwarding entrega el resultado de la ALU a la instrucción siguiente sin esperar a WB [2, 3].

Resolver por software duplica el tiempo. Sin hardware, la misma cadena exige 14 `nop` por vuelta, 28003 en total con los del final. La diferencia de ciclos entre los dos procesadores es $56011 - 28010 = 28001 = 28003 - 2$, es decir, un ciclo por cada `nop` menos los 2 ciclos del atasco del `ecall`, que solo ocurre con hazard unit. Con el mismo reloj, el procesador con hazard unit es

$$\frac{56011}{28010} \approx 2{,}00 \tag{26}$$

veces más rápido.

El CPI vuelve a ser engañoso. La versión con `nop` tiene mejor CPI (1.08 contra 1.17), pero tarda el doble. Contando solo las 24006 instrucciones útiles, su CPI efectivo es

$$\frac{56011}{24006} \approx 2{,}33, \tag{27}$$

frente a 1.17 con hazard unit. Esto es consecuencia de la ecuación de rendimiento: al usar `nop` suben las instrucciones y baja el CPI, pero el producto $N_{\text{instr}} \times \text{CPI}$, que determina el tiempo, es casi el doble [2, 3].

Cómo se podría reducir el costo. Si el cálculo real admitiera dos cadenas independientes, se podrían intercalar sus instrucciones: mientras una espera a su dependencia, se ejecuta una instrucción de la otra. Con dos cadenas intercaladas bastaría 1 `nop` por par, y con tres no harían falta, porque la distancia entre instrucciones dependientes llegaría a 3. Es la misma idea de la planificación de instrucciones que hacen los compiladores [3].

4.4. Comparación de los tres aspectos

La Tabla 10 resume los tres programas en los dos procesadores.

Programa	Procesador	Ciclos	Instrucciones	CPI	IPC
Accesos a memoria	Sin hazard (con `nop`)	30013	26007	1.15	0.867
	Con hazard	18013	14007	1.29	0.778
Saltos	Sin hazard (con `nop`)	27011	19009	1.42	0.704
	Con hazard	19013	11009	1.73	0.579
Operaciones entre registros	Sin hazard (con `nop`)	56011	52009	1.08	0.929
	Con hazard	28010	24006	1.17	0.857

Tabla 10: Comparación de los tres programas en los dos procesadores.

Con hazard unit, el CPI lo fija el control. En el procesador con hazard unit las dependencias de datos de los tres programas no producen atascos, y el CPI queda determinado por la fracción f de instrucciones que son saltos tomados:

$$\text{CPI} \approx 1 + 2f. \tag{28}$$

La Tabla 11 lo comprueba.

Programa	Saltos tomados	Instrucciones	f	$1 + 2f$ (CPI medido)
Accesos a memoria	2001	14007	0.143	1.29 (1.29)
Saltos	3999	11009	0.363	1.73 (1.73)
Operaciones entre reg.	1999	24006	0.083	1.17 (1.17)

Tabla 11: Relación entre el CPI con hazard unit y la fracción de saltos tomados.

El programa de saltos tiene el peor CPI porque más de un tercio de sus instrucciones son saltos tomados, y el de operaciones entre registros el mejor, porque casi todas sus instrucciones son operaciones de ALU que fluyen sin penalización.

Sin hazard unit, el costo lo fijan las dependencias. La relación entre los ciclos sin y con hazard unit es 1.67 en accesos a memoria, 1.42 en saltos y 2.00 en operaciones entre registros. Como cada `nop` agrega un ciclo, esa razón crece con la cantidad de `nop` que exige el programa: 12000 en accesos a memoria, 8000 en saltos y 28000 en operaciones entre registros, que es el programa más dependiente (cada instrucción usa el resultado de la anterior). En todos los casos, el CPI de la versión con `nop` parece mejor y el tiempo es peor, así que la comparación debe hacerse en ciclos totales.

Ninguno de los dos elimina el costo de los saltos. Los saltos tomados cuestan 2 ciclos en cualquier procesador y programa. Es un hazard de control, que ni el forwarding ni los `nop` resuelven, y por eso requiere técnicas como la predicción de saltos [3, 4].

5. Conclusiones

- **Parte 1 (accesos a memoria).** El programa realiza 4000 accesos a la memoria de datos (2000 escrituras y 2000 lecturas) en ejecuciones de 14007 y 26007 instrucciones, y la memoria queda con los valores esperados en ambos procesadores.
- La cuantificación analítica coincide con Ripes: las instrucciones y los ciclos reportados se ajustan exactamente al modelo $C = N + 4 + 2N_{\text{saltos}}$.
- Resolver los hazards de datos por software mantiene el mismo número de accesos a memoria de datos, pero aumenta las instrucciones un 86 % y los ciclos un 67 %, mientras que el forwarding los resuelve sin costo en ciclos para este programa.
- El CPI por sí solo puede ocultar el costo de los `nop`, por lo que la comparación debe hacerse en ciclos totales.
- Los saltos tomados impusieron 4002 ciclos en ambos procesadores, lo que muestra que los hazards de control no los resuelve el forwarding.
- **Parte 2 (saltos).** El programa ejecuta 5000 saltos, de los cuales 3999 son tomados y 1001 no tomados, y los contadores confirman 1000 vueltas por cada camino. Los ciclos medidos se ajustan exactamente a $C = N + 4 + 2N_{\text{tomados}} + S_{\text{ecall}}$: solo los saltos tomados tienen costo, 2 ciclos cada uno, y representan el 42 % del tiempo en el procesador con hazard unit.
- El `ecall` de salida se atiende en EX con hazard unit, con un atasco de hasta 2 ciclos, y en ID sin ella, donde puede ejecutarse desde el camino equivocado de un salto si `a7` ya está cargado.
- **Parte 3 (operaciones entre registros).** El programa ejecuta 16000 operaciones de tipo R (8 por vuelta, 2000 vueltas) encadenadas por dependencias RAW. Con hazard unit se retiran 24006 instrucciones en 28010 ciclos; sin ella, 52009 instrucciones en 56011 ciclos.
- El forwarding resuelve las 14000 dependencias de la cadena sin ningún ciclo extra, mientras que resolverlas por software exige 28003 `nop`, lo que duplica los ciclos (factor 2.00). El CPI efectivo sobre las instrucciones útiles es 1.17 con hazard unit y 2.33 sin ella.
- Una variante sin dependencias entre las operaciones necesita solo 3 `nop` y 28011 ciclos en el procesador sin hazard unit, lo que muestra que el costo proviene de las dependencias y no de las operaciones, y que la planificación de instrucciones puede reducirlo.
- **Conclusión general.** Los tres programas confirman que cada mecanismo cubre un tipo de riesgo distinto: el forwarding elimina el costo de las dependencias de datos entre instrucciones de ALU y los `nop` lo reemplazan a un costo de ciclos proporcional al número de dependencias, mientras que los saltos tomados cuestan siempre 2 ciclos. Con hazard unit, el CPI se ajusta a $1 + 2f$; sin ella, el tiempo crece un ciclo por cada `nop` (1.67, 1.42 y 2.00 veces más ciclos en las Partes 1, 2 y 3).

Referencias

[1] M. B. Petersen, "Ripes: A Visual Computer Architecture Simulator," en *2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE)*, 2021, pp. 1–8.

[2] S. L. Harris y D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[3] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2017.

[4] J. E. Smith, "A study of branch prediction strategies," en *Proc. 8th Annual Symposium on Computer Architecture (ISCA)*, 1981, pp. 135–148.

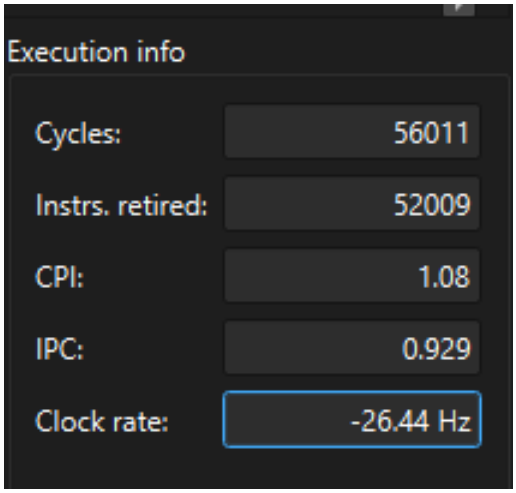
[5] A. Waterman y K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International, documento versión 20191213, 2019.

[6] M. B. Petersen, "Ripes: A graphical processor simulator and assembly editor for the RISC-V ISA," repositorio de GitHub. [En línea]. Disponible: `https://github.com/mortbopet/Ripes`

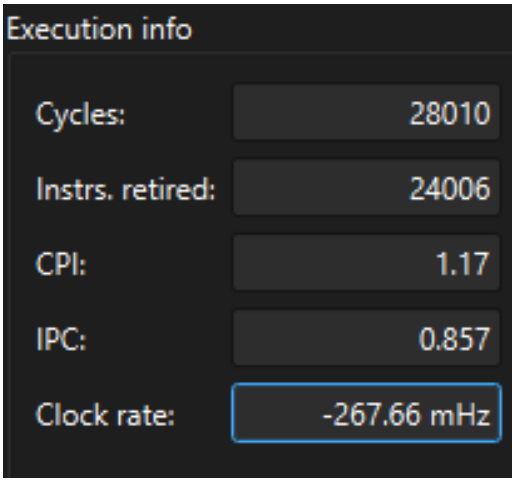
6. Anexos

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7ffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x000007d3
x6	t1	0x00000fa5
x7	t2	0x000007d0
x8	s0	0x00000000
x9	s1	0x00000000
x10	a0	0x00000000
x11	a1	0x00000000
x12	a2	0x00000000
x13	a3	0x00000000
x14	a4	0x00000000
x15	a5	0x00000000
x16	a6	0x00000000
x17	a7	0x0000000a
x18	s2	0x000007d0
x19	s3	0x00001775
x20	s4	0x00000fa3
x21	s5	0x00000fa3
x22	s6	0x00000ff3
x23	s7	0x00000050
x24	s8	0x01400000
x25	s9	0x00280000
x26	s10	0x00000000

Figura 5: Registros de Ripes al concluir el programa de operaciones entre registros.



(a) Sin hazard (con `nop`)



(b) Con hazard (original)

Figura 6: Execution info de Ripes para el programa de operaciones entre registros. El campo Clock rate no se utiliza en el análisis.